

Génie logiciel

Livrable 2

CESI 
ÉCOLE D'INGÉNIEURS



ProSoft

Génie Logiciel

Réalisé par

Maéva CHALLIES

Allan BOUHAMED

Enzo CADIÈRE

STOFFEL Maxime

Mathéo CARVAL

Philippe LUU

Projet encadré par

Ali SKAF

Année universitaire 2025-2026

Livrable 2 - 13/02/2026

Informations générales

Versions du document :

Ce tableau retrace toutes les versions du document :

Version	Date	Auteur(s)	Objet
1.0	12/02/2026	LUU Philippe, CARVAL Mathéo, CHALLIES Maéva, CADIÈRE Enzo, STOFFEL Maxime, BOUHAMED Allan	Création du document
1.1	13/02/2026	CARVAL Mathéo, CHALLIES Maéva, CADIÈRE Enzo	Revue du document

Liste de diffusion :

Ce tableau répertorie tous les destinataires ayant accès au document :

Entreprise / Organisme	Destinataire(s)	Contacts	Rôle
ProSoft	LUU Philippe	philippe.luu@prosoft.fr	Ingénieur Chef de projet & Scrum Master
ProSoft	CARVAL Matheo	matheo.carval@prosoft.fr	Ingénieur Architecte développement (Lead Dev)
ProSoft	CADIÈRE Enzo	enzo.cadiere@prosoft.fr	Ingénieur DevOps & CI
ProSoft	CHALLIES Maéva	maeva.challies@prosoft.fr	Ingénieure Qualité & Product Owner (Co-Lead Dev)
ProSoft	BOUHAMED Allan	allan.bouhamed@prosoft.fr	Ingénieur d'Études et Développement C#
ProSoft	STOFFEL Maxime	maxime.stoffel@prosoft.fr	Ingénieur d'Études et Développement C#
CESI	AOUAM Djamel	daouam@cesi.fr	Directeur informatique
CESI	SKAF Ali	askaf@cesi.fr	Responsable informatique

Synthèse Managériale

Ce document présente le bilan technique, fonctionnel et financier de la version 2.0 du logiciel de sauvegarde EasySave, développé par l'équipe d'ingénierie logicielle pour le compte de ProSoft. Ce second livrable répond à un double objectif stratégique qui est de démocratiser l'usage du logiciel grâce à une interface graphique (GUI) moderne basée sur le framework Avalonia UI, tout en intégrant des fonctionnalités de sécurité critiques, telles que le chiffrement via CryptoSoft, désormais exigées par le marché.

La conformité et l'évolution fonctionnelle constituent les premiers points clés de ce jalon. Le logiciel respecte l'intégralité du cahier des charges mis à jour et la transition vers une interface graphique intuitive permet désormais la gestion d'un nombre illimité de travaux de sauvegarde. Le paramétrage centralisé, incluant la gestion de la langue, des extensions de fichiers et des logiciels métiers, ainsi que la visualisation ergonomique des logs aux formats JSON ou XML, rendent le produit pleinement accessible aux utilisateurs non-techniciens.

Sur le plan architectural, l'adoption du pattern Model-View-ViewModel (MVVM) et le maintien d'une structure modulaire stricte, séparant le cœur applicatif de l'interface, garantissent une dette technique minimale. Cette architecture pérenne valide la capacité du logiciel à supporter les futures évolutions, notamment le parallélisme prévu pour la version 3.0, sans nécessiter de refonte du code existant déjà éprouvé.

En matière financière, la maîtrise des coûts et la rentabilité du projet sont assurées grâce à la réutilisation du moteur existant et à l'efficacité de l'équipe durant la phase de design, ce qui a permis de contenir l'investissement. Le coût total cumulé de R&D, englobant les versions 1 et 2, s'élève à 43 150,80 €. Avec un prix de vente unitaire fixé à 200 €, le retour sur investissement (ROI) est atteignable dès la 216ème licence vendue. Cet objectif est réaliste compte tenu de l'élargissement significatif du marché cible offert par l'apport de l'interface graphique.

Enfin, la qualité et l'industrialisation du projet sont garanties par le respect strict des normes de développement (DRY, nommage standardisé, usage de l'anglais), une couverture de tests unitaires sur le backend et une chaîne d'intégration continue (CI/CD) opérationnelle. Une documentation technique exhaustive accompagne le livrable pour assurer une reprise immédiate et fluide par n'importe quelle équipe de ProSoft. En conclusion, EasySave version

2.0 transforme un moteur technique performant en un produit commercialisable, stable et sécurisé, prêt pour le déploiement en environnement professionnel et les futures itérations de performance.

Sommaire

Informations générales.....	3
Versions du document :	3
Liste de diffusion :	3
Synthèse Managériale.....	4
Sommaire.....	6
1. Contexte et présentation.....	8
1.1 Présentation de ProSoft et du projet EasySave.....	8
1.2 Objectifs & contraintes (cahier des charges).....	8
1.3 Politique tarifaire et maintenance.....	9
2. Gestion de projet et environnement.....	11
2.1 Choix du framework utilisé (WPF, MAUI, Avalonia...).....	11
2.2 Gestion Git et versioning.....	14
2.3 Workflow Git.....	15
2.4 Suivi du projet.....	15
2.5 Maîtrise d'oeuvre.....	20
2.6 CI/CD.....	22
3. Architecture logicielle & choix techniques.....	23
3.1 Architecture globale.....	23
3.2 Architecture MVVM (version 2.0).....	24
3.3 Architecture de la DLL EasyLog.....	25
3.4 Intégration CryptoSoft.....	26
3.5 Gestion des paramètres généraux.....	27
3.6 Design patterns utilisés.....	28
4. Spécifications fonctionnelles (V3).....	30
4.1 Interface graphique.....	30
4.2 Gestion des travaux de sauvegarde.....	31
4.3 Gestion du cryptage.....	33
4.4 Gestion du fichier log.....	34
4.5 Gestion du fichier d'état.....	36
5. Diagrammes UML.....	39
5.1 Diagramme de classes.....	39
5.2 Diagramme de cas.....	40
5.3 Diagramme de séquence.....	40
5.4 Diagramme d'activité.....	42
7. Table des figures.....	43
8. Glossaire.....	44
9. Annexes.....	47
9.1 Guide d'installation et Support Technique (Détails ProSoft).....	47
9.2 Release note EasySave.....	47

9.3 Lien vers le dépôt GitHub.....	47
------------------------------------	----

1. Contexte et présentation

1.1 Présentation de ProSoft et du projet EasySave

ProSoft est un éditeur de logiciels reconnu, spécialisé dans le développement de solutions informatiques professionnelles. Dans le cadre de l'élargissement de son portefeuille de produits, l'entreprise lance le projet "EasySave", une solution logicielle de sauvegarde destinée à une clientèle B2B (Business to Business).

ProSoft a pour ambition de fournir un outil fiable, performant et simple d'utilisation pour sécuriser les données critiques des entreprises. Le logiciel EasySave doit permettre la gestion de sauvegardes complètes et différentielles, tout en assurant une traçabilité rigoureuse des opérations via des journaux d'événements. Ce projet s'inscrit dans une démarche évolutive, commençant par une version console (V1.0) pour valider le moteur de sauvegarde, avant d'évoluer vers une interface graphique moderne (V2.0) et des fonctionnalités avancées de parallélisme (V3.0).

1.2 Objectifs & contraintes (cahier des charges)

Le mandat confié à l'équipe de développement pour ce second livrable concerne l'évolution du logiciel EasySave vers les versions 1.1 et 2.0, avec pour objectif principal d'améliorer l'ergonomie, la flexibilité et la conformité aux nouveaux besoins exprimés par les clients, tout en conservant la compatibilité avec l'architecture existante. La version 2.0 met de côté le mode console au profit d'une interface graphique développée en C# avec le framework .NET Avalonia, reposant sur une architecture compatible avec le modèle MVVM afin de garantir la séparation claire entre l'interface utilisateur, la logique métier et la gestion des données. Cette transition impose une conception logicielle modulaire permettant de limiter les duplications de code et de faciliter les évolutions futures.

Le logiciel doit désormais permettre la gestion d'un nombre illimité de travaux de sauvegarde, chaque travail conservant les caractéristiques fondamentales définies dans la version précédente, notamment un nom unique, un répertoire source, un répertoire cible et un type de sauvegarde complète ou différentielle. L'utilisateur doit également disposer d'un accès aux paramètres généraux du logiciel via l'interface graphique, lui permettant notamment de définir la liste des extensions de fichiers devant être soumises au processus

de cryptage ainsi que le nom du logiciel métier dont la détection doit suspendre l'exécution des sauvegardes. Cette capacité de configuration centralisée constitue une exigence essentielle afin de garantir l'adaptabilité du logiciel aux différents environnements clients.

Une nouvelle contrainte fonctionnelle impose l'intégration du logiciel externe CryptoSoft, utilisé pour chiffrer les fichiers correspondant aux extensions définies par l'utilisateur. Le logiciel EasySave doit être capable de déclencher ce processus de chiffrement de manière transparente lors de la sauvegarde et de mesurer le temps nécessaire à cette opération. Cette information doit être enregistrée dans le fichier log journalier, qui conserve son implémentation sous forme de bibliothèque externe EasyLog.dll afin de garantir sa réutilisabilité et sa compatibilité avec les versions antérieures. Par ailleurs, afin de répondre aux besoins spécifiques de certains clients, la version 1.1 introduit la possibilité de choisir le format du fichier log entre JSON et XML. Cette fonctionnalité doit être pleinement intégrée dans la version 2.0 sans remettre en cause les mécanismes existants.

Le logiciel doit également être capable de détecter la présence d'un logiciel métier en cours d'exécution sur le système. Dans ce cas, il est interdit de lancer un nouveau travail de sauvegarde. Si une sauvegarde est déjà en cours, le logiciel doit terminer le traitement du fichier actif avant d'interrompre l'exécution, et cet événement doit être enregistré dans le fichier log afin de garantir la traçabilité des opérations. Cette contrainte vise à préserver les performances des applications critiques utilisées par le client et à éviter toute dégradation du système.

Les exigences non fonctionnelles restent strictement alignées avec les standards imposés par ProSoft. Le logiciel doit rester accessible aux utilisateurs francophones et anglophones grâce à un mécanisme de gestion multilingue. L'ensemble des fichiers de journalisation et d'état doit conserver un format structuré et lisible, facilitant leur exploitation par les équipes de support. Le code source doit respecter les conventions de nommage, limiter la duplication et garantir une maintenabilité élevée.

1.3 Politique tarifaire et maintenance

EasySave n'est pas seulement un défi technique, c'est un produit commercial soumis à une logique de rentabilité précise. La stratégie de tarification fixée par ProSoft établit un prix unitaire de 200 € HT par licence. À cela s'ajoute un contrat de maintenance facturé annuellement à hauteur de 12 % du prix d'achat, incluant les mises à jour mineures et

majeures. Le support technique étant assuré sur une plage horaire standard de 8h à 17h, cinq jours sur sept, il est essentiel de fournir une documentation technique et utilisateur irréprochable, ainsi que des fichiers de logs précis. Cette rigueur a pour objectif de réduire le temps de traitement des tickets support et d'augmenter la marge sur le contrat de support.

2. Gestion de projet et environnement

2.1 Choix du framework utilisé (WPF, MAUI, Avalonia...)

Le développement de la version 2.0 d'EasySave a nécessité le choix d'un framework graphique adapté aux contraintes du projet. L'application devait mettre de côté l'interface console de la v1.0 pour une interface graphique moderne tout en maintenant la compatibilité avec l'écosystème .NET 8 et en respectant les ressources limitées d'une équipe de six étudiants ingénieurs sur un cycle de développement de quelques mois.

L'étude comparative a porté sur trois frameworks majeurs de l'écosystème .NET :

- WPF
- .NET MAUI
- Avalonia.

Ces trois technologies proposent des approches différentes pour construire des interfaces graphiques multiplateformes avec du code C# et du XAML.

WPF reste la technologie de référence pour les applications de bureau Windows. Microsoft le maintient activement depuis 2006 et la documentation disponible est abondante. Le framework offre une maturité indéniable avec des composants éprouvés et une communauté importante sur Stack Overflow et les forums Microsoft. L'architecture MVVM s'intègre naturellement grâce au data binding performant et aux commandes natives. La courbe d'apprentissage est raisonnable pour des développeurs déjà familiers avec C# et XAML. Néanmoins, WPF présente des limitations importantes pour notre projet. Le framework fonctionne exclusivement sur Windows et nécessite le .NET Framework ou .NET Core 3.0+ sous Windows uniquement. Cette restriction à une plateforme unique pose problème si le client souhaite déployer l'application sur des serveurs Linux dans le futur. De plus, WPF montre son âge avec des composants au design daté qui nécessitent un effort conséquent de personnalisation pour obtenir une interface moderne. Le framework utilise également DirectX pour le rendu ce qui peut poser des problèmes de compatibilité sur certaines configurations matérielles anciennes ou virtualisées.

.NET MAUI représente la stratégie actuelle de Microsoft pour le développement multiplateforme. Le framework promet une base de code unique pour Windows, macOS, iOS et Android avec un accès aux API natives de chaque plateforme. L'architecture est moderne et tire parti des dernières fonctionnalités de C# 11 et .NET 8. Microsoft investit massivement dans MAUI avec des mises à jour régulières et une roadmap claire. Le support du Hot Reload accélère le développement en permettant de voir les modifications de l'interface en temps réel sans recompiler. L'intégration avec Visual Studio 2022 est excellente avec des designers visuels et des outils de débogage spécifiques. Cependant, MAUI s'avère inadapté à notre projet pour plusieurs raisons. Le framework cible principalement les applications mobiles et les interfaces tactiles. Les composants desktop restent limités et empruntent fréquemment des paradigmes mobiles peu adaptés à une utilisation bureau avec souris et clavier. La documentation spécifique au développement desktop est encore incomplète et se concentre majoritairement sur iOS et Android. MAUI souffre également de problèmes de stabilité sur Windows avec des bugs connus dans les contrôles WinUI3 sous-jacents. La courbe d'apprentissage est raide car le framework introduit de nouveaux concepts comme les handlers et les shell qui diffèrent significativement de WPF ou WinForms. Enfin, la taille des binaires produits est importante avec plusieurs dizaines de mégaoctets même pour des applications simples à cause de l'inclusion des runtime pour toutes les plateformes.

Avalonia se positionne comme une alternative open-source multiplateforme inspirée de WPF. Le framework reproduit l'API WPF en l'adaptant pour fonctionner sur Windows, macOS et Linux grâce à SkiaSharp pour le rendu graphique. Cette familiarité avec WPF réduit considérablement la courbe d'apprentissage pour des développeurs ayant des bases en XAML. Les composants fournis couvrent l'essentiel des besoins d'une application de bureau avec des contrôles classiques comme les grilles, listes, boutons, menus et dialogues. Le support du MVVM est natif et bien documenté avec des exemples d'implémentation de `INotifyPropertyChanged`, `ICommand` et `ObservableCollection`. Avalonia génère des binaires relativement légers comparé à MAUI et permet le déploiement self-contained où toutes les dépendances sont incluses dans un dossier portable. La communauté active sur GitHub et Discord répond rapidement aux questions et partage des solutions aux problèmes courants. Le projet bénéficie du sponsoring de plusieurs entreprises dont JetBrains ce qui garantit une certaine pérennité. La documentation officielle couvre bien les scénarios courants avec des guides pas à pas et des exemples de code. Les performances sont correctes grâce à l'utilisation de SkiaSharp qui tire parti de

l'accélération GPU quand disponible. Avalonia présente toutefois quelques inconvénients à considérer. La communauté reste plus petite que celle de WPF ou MAUI ce qui signifie moins de bibliothèques tierces disponibles et moins de réponses sur Stack Overflow. Certaines fonctionnalités avancées de WPF ne sont pas encore implémentées comme les effets visuels complexes ou les animations 3D. Les outils de développement sont moins intégrés dans Visual Studio que WPF avec un designer XAML qui nécessite une extension et reste basique. Quelques bugs mineurs subsistent dans les contrôles moins utilisés et peuvent nécessiter des contournements. La version stable actuelle est la 11.x mais l'API évolue encore régulièrement ce qui peut casser la compatibilité entre versions majeures.

Le choix d'Avalonia pour le projet EasySave découle de plusieurs facteurs techniques et pratiques. La multiplateforme constitue un avantage stratégique même si le déploiement initial cible uniquement Windows. Le client pourrait souhaiter à l'avenir exécuter l'application sur des serveurs de sauvegarde Linux sans avoir à redévelopper l'interface. La similarité avec WPF permet de capitaliser sur les connaissances acquises durant notre formation qui couvre extensivement le XAML et le pattern MVVM. L'équipe peut se référer à la documentation WPF pour comprendre des concepts complexes puis les adapter à Avalonia. Le poids des binaires reste raisonnable pour une application de sauvegarde qui sera déployée sur quelques postes seulement plutôt que distribuée massivement. La stabilité du framework suffit pour notre cas d'usage qui n'exploite pas de fonctionnalités exotiques mais se concentre sur des composants de base comme des listes, formulaires et barres de progression. Le développement d'une interface avec Avalonia s'apparente fortement à celui avec WPF ce qui réduit les risques et permet de livrer dans les délais impartis.

L'environnement de développement complet s'articule autour de Visual Studio 2022 Community Edition comme IDE principal. Cet environnement offre tous les outils nécessaires pour un projet .NET moderne : IntelliSense pour l'auto-complétion du code, débogueur intégré avec points d'arrêt et inspection des variables, gestionnaire NuGet pour les dépendances, Git intégré pour le contrôle de version. L'extension Avalonia for Visual Studio 2022 ajoute le support minimal du XAML avec coloration syntaxique et validation basique. Le previewer Avalonia permet de visualiser l'interface en cours de développement sans lancer l'application complète. ReSharper de JetBrains n'est pas utilisé car la licence commerciale dépasse le budget mais Visual Studio inclut déjà la plupart des fonctionnalités essentielles.

Le SDK .NET 8 fournit le runtime et les outils de compilation. Cette version LTS garantit un support à long terme jusqu'en novembre 2026 ce qui couvre largement la durée de vie prévisible du projet. Les nouvelles fonctionnalités de C# 12 comme les primary constructors et les collection expressions simplifient certaines parties du code. Le compilateur Roslyn génère des binaires optimisés pour x64 avec ahead-of-time compilation quand nécessaire.

Les packages NuGet essentiels incluent Avalonia 11.0 et ses dépendances comme Avalonia.Desktop pour le support Windows/Linux/macOS, Avalonia.Themes.Fluent pour un thème moderne proche de Windows 11, et Avalonia.ReactiveUI pour faciliter l'implémentation du MVVM.

2.2 Gestion Git et versioning

Le projet EasySave utilise Git avec GitHub comme plateforme d'hébergement et de collaboration, assurant le suivi des modifications, la gestion des versions et le travail en équipe. Le dépôt est privé et les autorisations suivent le principe du moindre privilège, les développeurs créant des branches et soumettant des pull requests, tandis que les responsables valident et fusionnent.

La stratégie repose sur Git Flow, la branche main contient les versions stables destinées à la production, la branche develop regroupe les fonctionnalités en cours avec des versions de test, et des branches dédiées servent aux fonctionnalités, corrections, refactorisations et documentation. Les commits respectent la norme Conventional Commits et le versioning sémantique (MAJOR.MINOR.PATCH) est automatisé par le pipeline CI/CD selon les changements détectés.

Le workflow prévoit la création de branches depuis develop, des commits atomiques, une revue de code avec tests automatiques, puis la fusion par les responsables. Les branches principales sont protégées, les conflits sont résolus manuellement et chaque version génère automatiquement une release avec changelog et exécutables pour plusieurs systèmes, garantissant la traçabilité et la reproductibilité du projet.

2.3 Workflow Git

Le processus Git du projet EasySave combine Git Flow avec des adaptations propres au projet afin de maintenir un historique clair, faciliter la collaboration de l'équipe et garantir la qualité du code par des contrôles automatisés. Le cycle de développement commence par la création d'un ticket sur GitHub décrivant le besoin, puis le développeur crée une branche depuis develop, travaille localement avec des commits atomiques conformes aux Conventional Commits et pousse régulièrement ses modifications. Une fois la fonctionnalité terminée, une pull request vers develop est soumise avec description des changements et tests réalisés.

L'ouverture de la pull request déclenche automatiquement le pipeline CI/CD, qui vérifie la compilation, exécute les tests, contrôle le formatage du code et analyse les dépendances. En parallèle, l'équipe effectue une revue du code portant sur la lisibilité, la maintenabilité, la cohérence architecturale et la présence de tests. Après validation, un responsable technique fusionne les modifications par squash and merge, ce qui maintient un historique linéaire.

Les développeurs mettent régulièrement à jour leur branche avec develop afin de limiter les conflits et les résoudre localement. Les versions stables sont publiées par fusion de develop vers main après vérification complète des fonctionnalités, de la documentation et des tests, puis le CI/CD génère automatiquement la version, les artefacts et le journal des modifications. En cas de bug critique en production, un correctif est créé directement depuis main, validé en priorité, puis reporté sur develop afin de conserver la cohérence entre les branches.

2.4 Suivi du projet

Suite à la livraison et à la validation de la version 1.0 (Console), la dynamique du projet a évolué pour aborder deux axes parallèles : la maintenance évolutive du moteur existant et la conception majeure de l'interface graphique (V2.0).

Transition technique et choix du Framework Dans un premier temps, l'équipe a procédé à une itération rapide pour finaliser la version 1.1 (cf. release note en annexe). Cette phase a permis d'intégrer la gestion des logs au format XML et les premières briques de CryptoSoft

via notre workflow GitFlow habituel, sanctuarisant le moteur ("Core") avant de le coupler à une interface complexe.

Pour la version 2.0, une réévaluation technique a été nécessaire. Initialement orientés vers .NET MAUI, nous avons constaté que ce framework ciblait prioritairement les écosystèmes mobiles (iOS, Android). EasySave étant un utilitaire destiné aux environnements Desktop et Serveur (Windows, Linux), nous avons pivoté vers le framework Avalonia UI. Ce choix technique garantit une meilleure intégration système et une compatibilité multiplateforme plus cohérente avec notre cible.

Conception UX/UI (Maquettage) Avant le développement, nous avons organisé une phase intensive de brainstorming et de maquettage pour transformer les fonctionnalités brutes en une interface ergonomique pour des non-techniciens. Nous avons structuré l'application en onglets fonctionnels :

- Accueil (Dashboard) : Vision synthétique et lancement rapide.
- Paramètres : Configuration centralisée (Langue, CryptoSoft, Logiciels métiers).
- Logs/Historique : Visualisation lisible des fichiers JSON/XML.

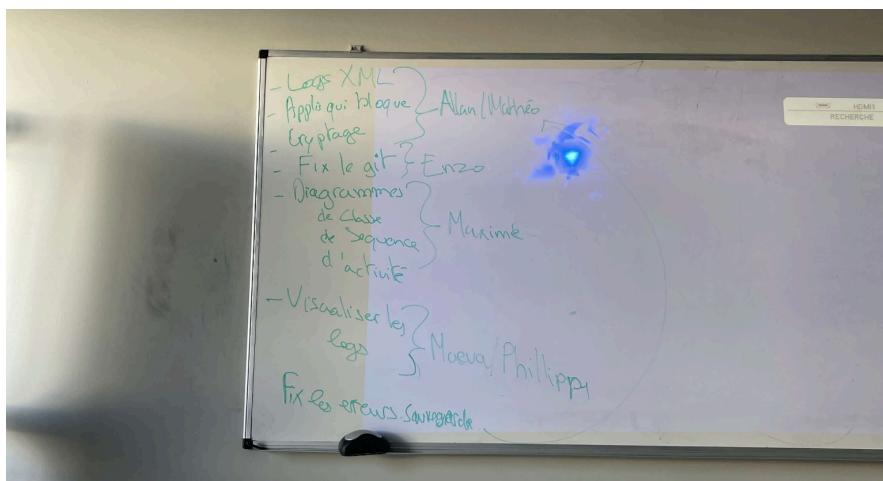


Figure 1 : Tableau répertoriant les tâches à réaliser pour le livrable 2.

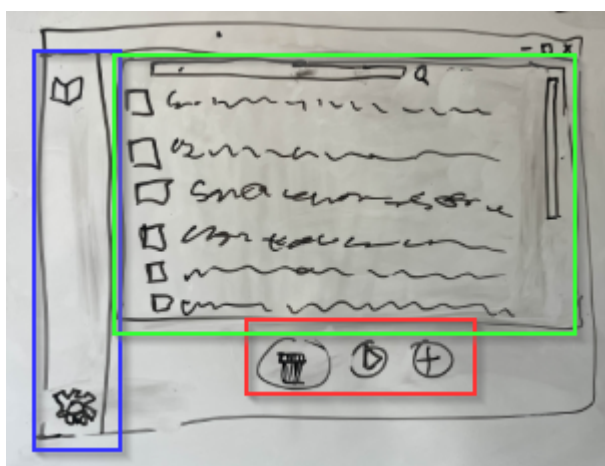


Figure 2 : Un croquis de la page d'accueil, en encadré bleu la side bar avec un bouton Home en haut et le bouton Settings en bas. En encadré vert, les tâches et un résumé de leur progression, état et informations sources et destinations avec la possibilité de les cocher, il est aussi possible de rechercher les tâches par nom et de les parcourir via "l'ascenseur". En encadré rouge les boutons d'actions permettant de supprimer, exécuter ou ajouter des tâches des sauvegardes.

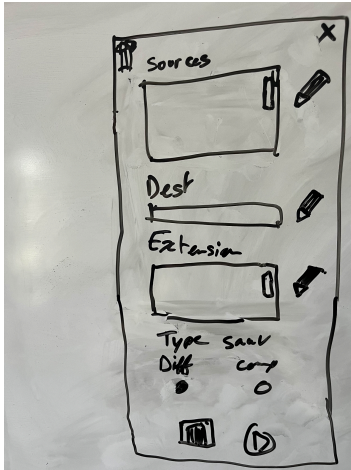


Figure 3 : Pop-up de création d'une tâche avec spécification des sources (possibilité de sélectionner les sources via explorateur de fichiers avec le crayon), de même pour la destination, les extensions de fichiers à sauvegarder, le type de sauvegarde, l'annulation de la création ou l'exécution de la tâche créée.

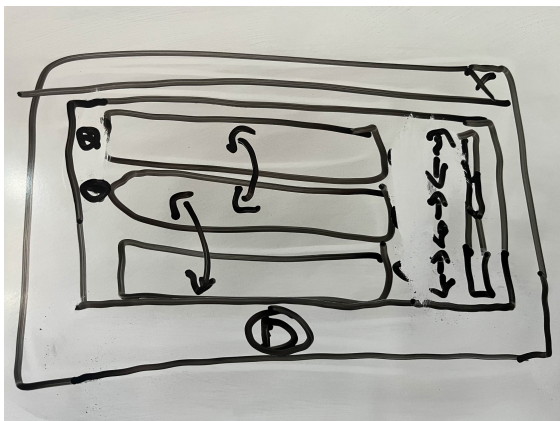


Figure 4 : Croquis de l'onglet de sélection des tâches, avec la possibilité de drag n drop les tâches afin de redéfinir l'ordre d'exécution de ces dernières.

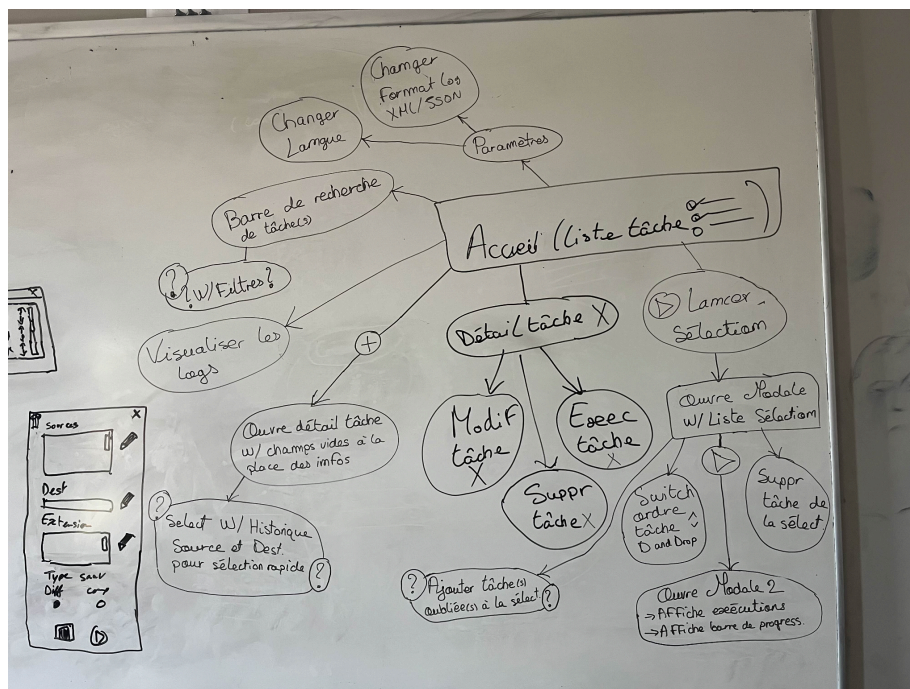


Figure 5 : Scénario des différents onglets et fonctionnalités.

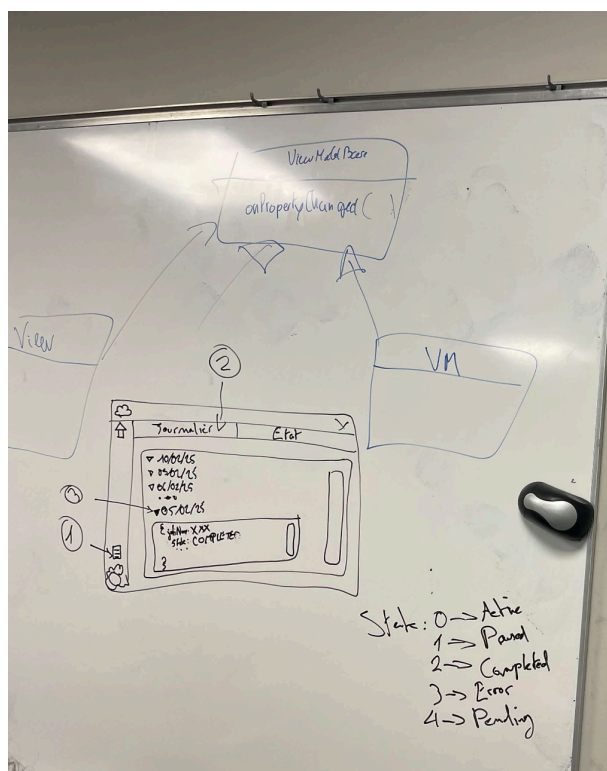


Figure 6 : Croquis de la partie des logs de l'application pour que l'utilisateur puisse visualiser les logs au lieu de se rendre dans AppData.

Ces maquettes ont servi de référence absolue pour les développeurs lors de l'intégration avec le framework Avalonia. Cela a permis de paralléliser le travail : pendant qu'un binôme travaillait sur le binding MVVM (le lien entre les données et la vue), un autre binôme se concentrait sur le design pur (XAML/CSS) pour reproduire fidèlement les maquettes validées.

Enfin, notre tableau Trello a été réorganisé pour refléter ce changement de paradigme. Les tâches purement algorithmiques ont laissé place à des "User Stories" orientées interface (ex: "En tant qu'utilisateur, je veux voir une barre de progression verte quand le backup est actif"), nous permettant de suivre l'avancement visuel du projet en temps réel.

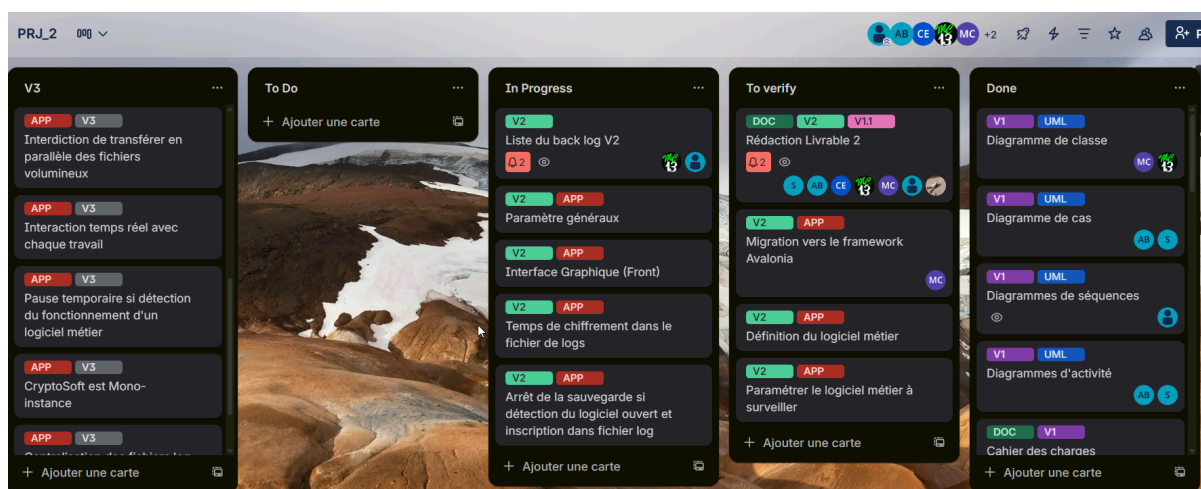


Figure 7 : Extrait du trello pour le livrable 2

2.5 Maîtrise d'oeuvre

L'évolution vers une interface graphique et l'intégration du chiffrement ont nécessité un investissement supplémentaire. Le coût de développement de ce second livrable a été calculé sur la base du volume horaire de l'équipe (Design, Apprentissage Avalonia, Implémentation).

Phase du projet	Entité	Coût entreprise	Volume	Coût total
Livrable 1	Ingénieur développement C# (x6)	18.68€/heure	350h	= 6538 x 6 (Ingénieurs) = 39 228€

Livrable 2 - 13/02/2026

Livable 2	Ingénieur développement C# (x6)	18.68€/heure	35h	653.8 x 6 = 3 922.8€
Total	XX	XX	XX	43 150.8€

Figure 8 : Tableau de chiffrage du livrable 1 et 2.

Le coût total du projet s'élève désormais à 43 150.8€. Avec un prix de vente unitaire de 200 € HT :

Seuil de rentabilité = $43\,150.8\text{ €} / 200\text{ €} \sim 216$ Licences.

Bien que l'investissement initial soit plus élevé, le passage à l'interface graphique (GUI) est un levier de rentabilité majeur pour ProSoft :

1. Élargissement du marché (TAM) : Le logiciel devient accessible aux non-techniciens, multipliant les cibles commerciales potentielles.
2. Réduction du TCO (Total Cost of Ownership) : L'ergonomie de la V2.0 réduit drastiquement les erreurs de manipulation. Cela diminue la charge du support technique, permettant à ProSoft de maximiser sa marge sur les contrats de maintenance (12% du prix de vente).

2.6 CI/CD

Le projet EasySave repose sur l'intégration et le déploiement continus afin d'automatiser la construction, les tests, le contrôle qualité et la publication, ce qui améliore la fiabilité et réduit les erreurs humaines. La solution s'appuie sur GitHub Actions, qui permet l'exécution automatisée des workflows sur plusieurs systèmes d'exploitation. Deux workflows principaux sont utilisés : l'un s'exécute automatiquement lors des push et des pull requests vers develop ou main pour vérifier le code, compiler, tester et publier, tandis que l'autre permet de déclencher manuellement une version spécifique.

La pipeline commence par des contrôles de qualité du code, incluant la vérification du formatage et l'analyse des vulnérabilités des dépendances. Si ces vérifications réussissent, le projet est compilé en mode Release puis soumis à des tests unitaires, dont les résultats sont conservés pour analyse. Le système calcule ensuite automatiquement la version selon les règles du versioning sémantique à partir de l'historique Git, génère un tag et déclenche la phase de publication.

La publication produit des exécutables pour Windows, Linux et macOS, ainsi que des archives portables et des installateurs adaptés à chaque plateforme. Les versions sont ensuite diffusées sous forme de releases avec changelog et artefacts téléchargeables. Le pipeline intègre également des optimisations de performance (cache des dépendances, exécution parallèle des tâches) et des mécanismes de sécurité, comme la gestion des secrets, la restriction des permissions et l'analyse des paquets vulnérables.

3. Architecture logicielle & choix techniques

3.1 Architecture globale

L'architecture d'EasySave s'articule autour d'une séparation claire des responsabilités en trois couches qui interagissent selon des règles strictes de dépendances. Cette organisation facilite la maintenance et permet de faire évoluer chaque composant de manière indépendante.

La couche de présentation gère l'interaction avec l'utilisateur. La version 1.0 utilise une interface console structurée en composants réutilisables comme les formulaires de saisie, les dialogues de confirmation et les assistants de création de tâches qui s'appuient sur le `LocalizationService` pour le multilingue. La version 2.0 remplace cette interface par une interface graphique développée avec Avalonia qui suit le pattern MVVM.

La couche métier regroupe les services applicatifs. Le `BackupManager` orchestre la création, la suppression et l'exécution des travaux de sauvegarde en maintenant une liste interne de `BackupJob` et en vérifiant les contraintes comme la limite du nombre de travaux simultanés. Le `FileTransferService` effectue les opérations de copie en distinguant les sauvegardes complètes des sauvegardes différentielles, calcule la progression en temps réel et met à jour l'état via le `StateWriter`. Le `LocalizationService` charge les traductions depuis un fichier JSON embarqué comme ressource dans l'assembly.

La couche de persistance gère le stockage. Le `StateWriter` maintient un fichier `state.json` contenant l'état de tous les travaux en temps réel via un dictionnaire où la clé est le nom du travail et la valeur est un objet `StateEntry` sérialisé. Le logger écrit les entrées dans des fichiers journaliers au format JSON ou XML selon la configuration. La bibliothèque `EasyLog` encapsule toute la logique de journalisation pour la rendre réutilisable.

L'initialisation suit un ordre précis défini dans `Program.cs` : `LocalizationService` → `StateWriter` → `Logger` → `FileTransferService` → `BackupManager`. Cette construction par injection de dépendances permet de remplacer facilement une implémentation par une autre pour les tests. Les communications respectent le principe de dépendance unidirectionnelle : la présentation appelle le métier, le métier appelle la persistance, jamais l'inverse.

La gestion des erreurs utilise les exceptions .NET typées (ArgumentException, InvalidOperationException, DirectoryNotFoundException, IOException) capturées par la couche présentation pour afficher des messages compréhensibles. Les données transitent via des objets métier distincts : BackupJob pour la logique complète, BackupLogEntry pour les logs, StateEntry pour l'état simplifié. Cette séparation permet de faire évoluer chacun indépendamment.

3.2 Architecture MVVM (version 2.0)

Le pattern MVVM imposé par Avalonia sépare strictement la logique de présentation, la logique métier et l'interface utilisateur. Cette séparation améliore la testabilité du code et facilite les modifications de l'interface sans impacter le fonctionnement.

Le modèle BackupJob constitue le cœur de l'application avec ses propriétés (Name, SourcePath, TargetPath, BackupType, BackupState, TotalFiles, TotalSize, RemainingFiles, RemainingSize, Progress, CurrentSourceFile, CurrentTargetFile) et ses méthodes de modification d'état. UpdateProgress recalcule le pourcentage en fonction des octets restants. MarkAsCompleted passe l'état à COMPLETED avec progression à 100. MarkAsError positionne l'état ERROR et réinitialise les fichiers courants.

La vue correspond aux fichiers XAML qui décrivent l'interface graphique. MainWindow.xaml structure la fenêtre principale avec barre latérale de navigation et zone de contenu. Des vues spécialisées gèrent les pages (HomeView, SettingsView, LogsView). Les bindings connectent les éléments visuels aux propriétés du ViewModel : TextBlock affiche le nom via binding sur Name, ProgressBar affiche la progression via Progress, Button déclenche une commande via Command. La vue ne contient aucune logique métier dans son code-behind.

Le ViewModel fait le pont entre vue et modèle. MainViewModel agrège les ViewModels de pages et gère la navigation. HomeViewModel expose une ObservableCollection<BackupJobViewModel> pour afficher la liste des travaux. Quand l'utilisateur clique sur exécution, la vue appelle ExecuteJobCommand qui récupère le BackupJob et invoque ExecuteJob du BackupManager. Le ViewModel intercepte les événements pour mettre à jour ses propriétés.

L'implémentation d'`INotifyPropertyChanged` constitue le mécanisme central. Chaque `ViewModel` déclenche `PropertyChanged` quand une propriété change. Les bindings écoutent cet événement et mettent à jour l'affichage automatiquement. Les propriétés utilisent un pattern standard : champ privé, propriété publique vérifiant le changement, déclenchement de `PropertyChanged` si nécessaire.

Les commandes encapsulent les actions via `ICommand` (`Execute` et `CanExecute`). `RelayCommand` implémente `ICommand` avec des délégués. Le `ViewModel` expose des propriétés `ICommand` bindées aux boutons. Quand `CanExecute` retourne `false`, le bouton se désactive automatiquement. `ObservableCollection<T>` reflète automatiquement les changements de listes dans l'interface. La navigation utilise `CurrentViewModel` dans `MainViewModel` avec `ContentControl` et `DataTemplates` pour afficher automatiquement la vue correspondante.

L'architecture respecte plusieurs principes stricts : le `ViewModel` ne référence jamais la vue, la vue ne manipule jamais directement les modèles, les services métier ignorent totalement les `ViewModels`. Cette séparation garantit que la logique métier peut être testée sans instancier d'interface graphique.

3.3 Architecture de la DLL EasyLog

La bibliothèque `EasyLog` est un composant autonome structuré en couches respectant les principes SOLID. Son architecture modulaire permet d'ajouter de nouveaux formats sans modifier le code existant.

La couche d'abstraction définit les contrats. `ILogger<T>` déclare `Log<T>`, `LogCollection<T>`, `ReadLog<T>`, `SetOutputPath`, `SetFormat` et `Flush`. `ILogFormatter<T>` définit `Format`, `FormatCollection`, `Parse` et `ParseCollection`. Cette séparation entre logging et formatage applique le principe de responsabilité unique.

`LoggerBase` implémente la logique commune en gérant un dictionnaire de formatters indexé par type. `GetFormatter<T>` instancie `JsonFormatter` ou `XmlFormatter` selon la configuration si aucun formatter personnalisé n'existe. Le formatter créé est stocké dans le dictionnaire pour réutilisation. La méthode `Log` suit un algorithme en cinq étapes atomiques : récupérer formatter, lire fichier via `ReadFromFile`, parser contenu via `ParseCollection`, ajouter entrée, reformater collection, écrire via `WriteToFile`. Un verrou assure la thread-safety.

Les méthodes `WriteToFile` et `ReadFromFile` sont déclarées abstraites, appliquant le `Template Method` pattern. `JsonLogger` et `XmlLogger` héritent de `LoggerBase` et implémentent ces méthodes avec `File.WriteAllText` et `File.ReadAllText`. `JsonFormatter` utilise `System.Text.Json` avec options `WriteIndented`, `CamelCase` et `UnsafeRelaxedJsonEscaping`. `XmlFormatter` utilise `System.Xml.Serialization` avec structure `<Items><Item>` pour lisibilité dans Notepad.

Les exceptions spécifiques (`LoggerException`, `FormatterException`) facilitent le diagnostic avec messages contextuels incluant le type d'objet, l'opération échoue et l'exception sous-jacente. L'injection dans `FileTransferService` permet de logger chaque transfert via `BackupLogEntry` (timestamp, nom travail, chemins UNC, taille, temps). L'extensibilité permet d'ajouter CSV ou autres formats en créant simplement une classe implémentant `ILogFormatter<T>`.

3.4 Intégration CryptoSoft

CryptoSoft est un outil de chiffrement hybride combinant RSA-4096 et AES-256-GCM. Le schéma cryptographique génère une clé AES-256 session aléatoire unique par fichier, chiffre les données par chunks de 64 KiB en mode GCM (confidentialité + authentification), wrappe la clé session avec RSA-OAEP-SHA256, et produit un fichier avec header binaire suivi des chunks chiffrés.

Le format du fichier commence par un header de 533 octets : magic number "CSFT" (4 octets), version (2 octets), identifiant algorithme RSA-OAEP-SHA256 + AES-256-GCM (1 octet), longueur clé wrappée (2 octets), nonce 96 bits, clé session chiffrée RSA (512 octets pour RSA-4096). Après le header, chaque chunk contient : taille ciphertext (4 octets little-endian), ciphertext, tag GCM (16 octets). Un chunk de taille zéro signale la fin du flux.

L'intégration dans `FileTransferService` utilise une méthode `ShouldEncryptFile` vérifiant l'extension du fichier contre la liste configurée (.docx, .xlsx, .pdf). Si correspondance, `CryptoSoftApi.EncryptFile` est appelé avec trois paramètres : source, destination .enc, clé publique PEM. Le streaming par chunks via `FileStream` et buffer de 64 KiB limite la mémoire à 200 KiB constant. Chaque chunk utilise un nonce dérivé unique (base XOR compteur monotone).

La gestion des clés est asymétrique : clé publique distribuée avec l'application ou configurée dans les paramètres (format PEM avec "BEGIN PUBLIC KEY"), clé privée conservée par l'administrateur dans HSM ou coffre-fort sécurisé. La génération s'effectue en externe via "cryptosoft keygen --private private.key --public public.key" créant une paire RSA-4096.

Les performances restent acceptables grâce aux instructions AES-NI : un fichier d'1 GiB se chiffre en ~2 secondes sur processeur moderne, overhead 0.03% (header 533 octets + 20 octets par chunk de 64 KiB), opération RSA unique par fichier (5-10 ms). La sécurité repose sur GCM (confidentialité + authentification via tag 128 bits), nonces uniques par chunk (anti-rejeu), wrapping RSA-OAEP (liaison destinataire), et CryptographicOperations.ZeroMemory après usage.

Les limitations incluent l'absence de protection contre les ordinateurs quantiques (RSA et AES vulnérables aux algorithmes de Shor et Grover) et l'absence de forward secrecy (compromission clé privée expose tous les fichiers chiffrés avec la clé publique). Ces risques sont mitigés par rotation régulière des clés et stockage sécurisé dans HSM type Azure Key Vault ou HashiCorp Vault.

3.5 Gestion des paramètres généraux

EasySave doit garder des paramètres utilisateur pour adapter son comportement sans recompilation. Ces paramètres couvrent la langue de l'interface, le format des logs, les chemins des fichiers log et état, le chemin de l'outil de chiffrement et la liste des extensions à chiffrer, ainsi que la liste des applications bloquantes. Le stockage se fait dans un fichier Config.json placé dans le dossier AppData\Roaming\EasySave de l'utilisateur, ce qui respecte la convention Windows pour les données de configuration par utilisateur.

La classe Configuration représente ces paramètres sous forme d'objet .NET. Elle fournit des getters et setters avec une vérification simple, par exemple pour éviter des valeurs vides. Des valeurs par défaut sont définies dans le constructeur, ce qui garantit que l'application peut fonctionner même si le fichier de configuration n'existe pas ou est incomplet. Le chargement est assuré par ConfigurationManager qui suit un pattern Singleton. Au premier accès, il vérifie si Config.json existe et le crée si besoin. Il déserialise ensuite le JSON dans une structure temporaire, puis reconstruit l'objet Configuration avec des valeurs valides.

Dans la version actuelle, la langue est stockée dans Config.json avec un code long comme fr-FR ou en-US. Pour l'interface, SettingsViewModel transforme ce code en index de langue et applique un code court fr ou en au LocalizationService. Le format de log est stocké sous forme de chaîne, puis converti en enum LogFormat. Les chemins de log et d'état sont sauvegardés dans le fichier, mais si l'utilisateur fournit un dossier au lieu d'un fichier, SettingsViewModel adapte le chemin en ajoutant un nom de fichier par défaut. Les extensions à chiffrer et les applications bloquées sont stockées comme listes de chaînes.

L'édition se fait via la page de paramètres de l'interface Avalonia. Les contrôles sont liés aux propriétés de SettingsViewModel. Quand l'utilisateur modifie un champ, un indicateur de modifications non sauvegardées est actif. Le bouton Enregistrer appelle la méthode Save qui charge la configuration actuelle, met à jour toutes les valeurs, puis écrit le JSON avec ConfigurationManager.SaveConfiguration. La langue et le thème sont appliqués immédiatement à l'interface. A la fin de l'enregistrement, SettingsViewModel déclenche l'événement SettingsSaved, et MainViewModel recharge la liste des applications bloquées dans BackupManager pour que la nouvelle configuration soit prise en compte sans redémarrage.

La validation dans le code reste simple. Il n'y a pas de vérification avancée de permissions ou de création automatique de dossiers, mais les chemins sont adaptés si l'utilisateur saisit un dossier au lieu d'un fichier. Les extensions sont normalisées et nettoyées via la classe Configuration, qui ajoute le point si besoin et met en minuscules. Cette gestion reste suffisante pour assurer un comportement stable, mais peut être renforcée dans une évolution future.

3.6 Design patterns utilisés

Plusieurs patterns sont utilisés pour structurer l'application et limiter les dépendances directes. Le pattern Strategy apparaît dans EasyLog avec les formatters JsonFormatter et XmlFormatter qui implémentent la même interface. LoggerBase choisit le formatter selon le format configuré, ce qui permet de changer de format sans modifier la logique de log. Cette

séparation respecte bien le principe ouvert-fermé car on peut ajouter un nouveau format en ajoutant une classe de formatter.

Le pattern Repository est présent dans BackupManager. Il encapsule la gestion des BackupJob et expose les opérations CreateJob, GetJob, GetAllJobs, ModifyJob et DeleteJob. La collection interne reste privée, ce qui évite que la présentation modifie directement les données. Cela permet aussi de remplacer l'implémentation interne par une autre source de données sans changer l'interface du manager.

Le pattern Observer est utilisé de façon indirecte avec StateWriter. A chaque modification d'état pendant une sauvegarde, le FileTransferService appelle UpdateJobState et le fichier state.json est mis à jour. Des outils externes peuvent surveiller ce fichier pour suivre la progression. La communication n'est pas un observer classique en mémoire, mais le fichier joue le rôle de point d'observation central.

Le pattern Dependency Injection est appliqué dans Program.cs. Les services ne sont pas instanciés dans les classes qui les utilisent. Ils sont construits dans l'ordre puis passés aux constructeurs. Cela rend le code plus modulaire et facilite les tests en remplaçant un service par un mock. Cette approche reste simple mais elle respecte l'idée d'injection de dépendances.

Le pattern Template Method est appliqué dans LoggerBase. La méthode Log définit un algorithme complet, mais délègue la lecture et l'écriture des fichiers à ReadFromFile et WriteToFile. Ces méthodes sont implémentées différemment dans JsonLogger et XmlLogger. Le squelette reste commun, ce qui évite la duplication.

Le pattern Singleton est utilisé pour ConfigurationManager. Une seule instance est créée et partagée dans toute l'application. Ce choix simplifie l'accès à la configuration sans devoir passer l'instance partout, mais rend les tests un peu moins flexibles. Malgré cela, pour une application de cette taille, ce compromis reste acceptable.

4. Spécifications fonctionnelles (V3)

4.1 Interface graphique

L'interface graphique de la version 2.0 marque une rupture avec l'approche console de la v1.0. La fenêtre principale se divise en deux zones : une barre latérale de navigation à gauche (Accueil, Paramètres, Historique, Aide) et une zone de contenu principal affichant le contenu correspondant à la section sélectionnée.

La page d'accueil présente les travaux sous forme de cartes visuelles. Chaque carte affiche le nom en haut en police plus grande, une icône de type (disque plein pour sauvegarde complète, disque avec flèches pour différentielle), un statut via pastille colorée (vert=terminé, bleu=actif, gris=attente, orange=pause, rouge=erreur), les chemins abrégés avec infobulle au survol, et une barre de progression horizontale changeant de couleur selon l'état. Les interactions incluent des cases à cocher (individuelle + globale pour tout sélectionner), clic sur carte pour panneau détaillé montrant toutes les informations, et bouton de modification ouvrant un dialogue d'édition.

La barre d'outils regroupe les actions principales : bouton "Nouveau travail" (icône plus), bouton "Exécuter" (désactivé si aucune sélection), bouton "Supprimer" (avec confirmation), barre de recherche filtrant en temps réel sur le nom, et boutons de filtre rapide pour afficher uniquement les sauvegardes complètes, différentielles, ou dans un état particulier comme les erreurs.

Le dialogue de création utilise un formulaire vertical : champ texte pour nom (validation unicité, 50 caractères max, pas de caractères spéciaux Windows), section "Sources" avec liste dynamique et bouton "Ajouter une source" ouvrant un sélecteur de dossiers supportant disques locaux/réseau/UNC, champ "Destination" avec bouton de sélection et test d'écriture, menu déroulant pour type (complète vs différentielle) avec explication, boutons "Annuler" et "Créer".

La page de paramètres organise les options en sections clairement délimitées. Section "Apparence" : bouton bascule mode clair/sombre avec application immédiate. Section "Langue" : menu déroulant fr/en avec mise à jour instantanée de tous les textes. Section "Journalisation" : menu déroulant format JSON/XML, champs texte avec navigation pour

chemins logs et état (défaut AppData). Section "Chiffrement" : liste extensions éditables avec champ texte pour ajouter (.docx, .xlsx, .pdf), bouton pour retirer, bouton "Parcourir" pour sélectionner clé publique RSA. Section "Applications bloquées" : liste processus (excel.exe, word.exe) empêchant l'exécution des sauvegardes.

La page d'historique affiche un tableau chronologique inversé (entrées récentes en haut) avec colonnes (timestamp, travail, statut, fichiers, durée). Clic sur ligne pour développer les détails montrant tous les fichiers copiés (chemin source, destination, taille, temps transfert). Champ de recherche filtrant par travail/date, boutons de filtre rapide pour exécutions réussies/en erreur/type spécifique.

La page d'aide structure la documentation en sections thématiques dans une barre latérale : Démarrage rapide, Types de sauvegarde, Sources multiples, Ordre d'exécution, Recherche et filtres, Paramètres avancés, Consultation des logs, Bonnes pratiques. Le contenu s'affiche avec mise en forme claire (titres, paragraphes, captures d'écran).

Les notifications toast apparaissent en bas à droite : vertes pour succès (nom travail + nombre fichiers copiés), oranges pour avertissements (fichier non copié), rouges pour erreurs bloquantes (accès source impossible). Cliquables pour détails ou accès direct à l'historique. Disparition automatique après quelques secondes ou fermeture manuelle.

L'interface respecte les conventions Windows : raccourcis (Ctrl+N nouveau, Ctrl+S sauvegarder, Ctrl+F rechercher, F1 aide, F5 rafraîchir), menu contextuel clic droit (exécuter, modifier, supprimer, copier détails), survol changeant l'apparence des éléments interactifs, boutons désactivés grisés.

4.2 Gestion des travaux de sauvegarde

La création d'un travail commence par un nom unique validé lors de la saisie : pas de vide, pas d'espaces uniquement, pas de noms déjà utilisés, pas de caractères spéciaux Windows, maximum 50 caractères. Les sources multiples s'ajoutent via dialogue de sélection avec validation d'accès lecture : si répertoire inexistant, message d'erreur explicite ; si inaccessible (permissions), suggestion de vérifier droits. La version 2.0 ne limite plus le nombre de sources (v1.0 limitait à cinq).

La destination nécessite validation d'accès écriture via test de création puis suppression fichier temporaire. Le répertoire peut se situer sur même disque, disque différent, lecteur réseau ou partage UNC. Le système ne vérifie pas l'espace disponible à la configuration car variable entre création et exécution.

Le type de sauvegarde propose deux options. Complète : copie tous les fichiers à chaque exécution, garantit destination identique aux sources. Différentielle : copie uniquement fichiers nouveaux ou modifiés depuis dernière complète, compare date modification source/destination, économise temps et espace. Explication détaillée sous le menu pour aider le choix.

La modification utilise le même formulaire pré-rempli. Nom modifiable sauf si déjà utilisé. Sources ajoutables/retirables librement. Destination changeable. Type modifiable avec avertissement sur implications : complète → différentielle nécessite réexécution complète pour baseline ; différentielle → complète n'a pas d'impact immédiat mais prochaines exécutions copieront tout. Boutons "Enregistrer" et "Annuler".

L'exécution se déclenche via : bouton "Exécuter" (travaux sélectionnés avec dialogue de confirmation listant les travaux), bouton dans panneau détaillé (lance uniquement ce travail), ou clic droit + menu contextuel "Exécuter maintenant".

Le processus d'exécution vérifie d'abord les applications bloquées (Excel, Word configurés). Si détectées, annulation avec message. Sinon, statut → Active, barre de progression animée, itération sur toutes les sources, récupération liste fichiers récursivement, calcul taille/nombre total, copie avec vérification extension pour chiffrement éventuel via CryptoSoft, mise à jour state.json temps réel, notification finale de succès.

La gestion d'erreurs est résiliente. Échec sur un fichier (verrou, permission, erreur disque) : enregistrement dans log, exécution continue, compteur d'erreurs incrémenté. Fin avec erreurs : statut "Completed with errors", notification indiquant fichiers copiés + nombre d'erreurs, clic ouvrant historique filtré. Erreur fatale (accès source impossible) : arrêt immédiat, statut "Error", notification rouge.

La suppression nécessite double confirmation : dialogue listant travaux à supprimer rappelant action irréversible (fichiers déjà sauvegardés non affectés), bouton "Supprimer"

définitivement" + "Annuler". Une fois supprimé, travail disparaît, informations perdues, historique reste dans logs mais ne référence plus travail actif.

L'exécution séquentielle respecte l'ordre de sélection (A, C, B → exécution A, C, B). Un travail attend que le précédent se termine. Évite la contention disque, simplifie le suivi progression. Évolution possible pour exécution parallèle de travaux sans dépendances communes.

4.3 Gestion du cryptage

Configuration nécessite deux éléments : liste d'extensions (.docx, .xlsx, .pdf) normalisées automatiquement (point préfixé, minuscules), et fichier clé publique RSA via dialogue filtrant .pem/.key. Validation basique vérifie "BEGIN PUBLIC KEY" ou "BEGIN RSA PUBLIC KEY". Chemin affiché en lecture seule, clé doit rester accessible à cet emplacement. Génération externe via "cryptosoft keygen --private private.key --public public.key" créant RSA-4096. Fichier private.key conservé dans HSM/coffre-fort sécurisé, public.key distribué librement.

Application automatique durant exécution : vérification extension fichier, si correspondance invoque CryptoSoftApi.EncryptFile (source, destination .enc, clé publique). Streaming 64 KiB chunks : lecture FileStream, chiffrement AES-256-GCM avec clé session aléatoire, wrapping clé session avec RSA-OAEP-SHA256, écriture fichier avec header binaire + chunks chiffrés. Fichier source intact, seul fichier chiffré vers destination. Exemple : "rapport.docx" devient "rapport.docx.enc".

Progression affiche fichier en cours sous barre de progression avec estimation temps basée sur taille et vitesse observée (imprécise car varie selon charge CPU/disque). Erreurs non bloquantes : échec chiffrement enregistré dans log, exécution continue, notification finale mentionnant erreurs. Causes communes : clé corrompue, fichier inaccessible, disque plein.

Déchiffrement non implémenté dans EasySave (pas son rôle). Restauration externe via "cryptosoft decrypt --input rapport.docx.enc --output rapport.docx --privkey private.key" par administrateur possédant clé privée. Séparation garantit que système sauvegarde ne peut jamais déchiffrer données chiffrées.

Désactivation par vidage liste extensions : aucun fichier chiffré, tous copiés en clair.
Réactivation par ajout extensions. Performance acceptable : 1 GiB en 2 secondes
processeur moderne, overhead 0.03%, impact notable uniquement sur utilisation CPU
(processeur ancien/chargé peut ralentir système).

4.4 Gestion du fichier log

Le fichier log enregistre un historique détaillé de toutes les opérations de sauvegarde effectuées. Ces informations permettent de tracer les actions réalisées, de diagnostiquer les problèmes et de générer des rapports d'activité. La structure et le format du fichier log sont conçus pour être à la fois lisibles par un humain et analysables par des outils automatisés.

La structure du fichier log utilise un fichier journalier par jour d'exécution. Chaque fois qu'une sauvegarde s'exécute, les entrées de log sont ajoutées au fichier correspondant à la date actuelle. Le nom du fichier suit le format YYYY-MM-DD.json ou YYYY-MM-DD.xml selon le format configuré. Par exemple, les sauvegardes du 12 février 2026 génèrent des entrées dans le fichier 2026-02-12.json ou 2026-02-12.xml. Cette organisation par jour facilite l'archivage et la consultation. Un administrateur peut facilement retrouver les logs d'une période spécifique en naviguant dans les fichiers par date. Les fichiers anciens peuvent être archivés ou supprimés selon la politique de rétention de l'organisation.

Le format JSON du fichier log contient un tableau d'objets où chaque objet représente le transfert d'un fichier. Chaque entrée contient six champs. Le champ "timestamp" enregistre la date et l'heure précise du transfert au format ISO 8601 comme "2026-02-12T14:30:45.123Z". Le champ "backupName" contient le nom du travail de sauvegarde qui a effectué ce transfert. Ce champ permet de filtrer les logs par travail. Le champ "sourcePath" contient le chemin complet du fichier source au format UNC. Le format UNC garantit que les chemins sont interprétables même sur des systèmes distants. Par exemple, un chemin local "C:\Documents\rapport.docx" est converti en "\\localhost\C\$\Documents\rapport.docx". Le champ "targetPath" contient le chemin complet du fichier destination également au format UNC. Le champ "fileSize" enregistre la taille du fichier en octets. Ce champ permet de calculer la quantité totale de données sauvegardées. Le champ "transferTime" enregistre le temps de transfert en millisecondes. Ce champ permet d'analyser les performances et d'identifier les fichiers qui prennent beaucoup de temps à copier.

L'écriture dans le fichier log s'effectue de manière atomique pour garantir l'intégrité. Quand une nouvelle entrée doit être ajoutée, le système lit d'abord le fichier complet, déserialise le tableau JSON ou XML en objets, ajoute la nouvelle entrée au tableau, reserialise le tout et réécrit le fichier complet. Cette approche garantit que le fichier reste toujours dans un état cohérent. Si l'application se termine brutalement durant l'écriture, le fichier contient soit les entrées précédentes soit les entrées avec la nouvelle, mais jamais un état intermédiaire corrompu. Un verrou protège l'accès au fichier pour éviter les écritures concurrentes si plusieurs threads tentent de logger simultanément.

La gestion des erreurs durant la journalisation utilise une approche défensive. Si l'écriture dans le fichier log échoue à cause d'un disque plein, de permissions insuffisantes ou d'un autre problème, l'erreur est enregistrée dans la sortie d'erreur standard mais ne bloque pas l'exécution de la sauvegarde. Cette décision découle du fait que l'échec de journalisation ne doit pas empêcher la sauvegarde de se terminer. L'utilisateur est informé qu'un problème de journalisation est survenu via une notification. Le travail de sauvegarde continue et se termine normalement. Les fichiers sont bien copiés mais les détails de cette exécution ne sont pas enregistrés dans le log. Cette situation est rare en pratique mais doit être gérée proprement.

La consultation des logs s'effectue via l'interface graphique dans la page d'historique. Cette page lit tous les fichiers de log du répertoire configuré, les parse et affiche les entrées dans un tableau trié par date décroissante. L'utilisateur peut filtrer par travail, par date ou par statut. Un clic sur une entrée développe les détails montrant tous les fichiers traités durant cette exécution. Cette interface évite d'avoir à ouvrir manuellement les fichiers JSON ou XML pour consulter l'historique. Néanmoins, les fichiers restent accessibles directement pour permettre l'analyse avec des outils externes comme Excel, des scripts Python ou des systèmes de monitoring.

La rotation des logs n'est pas automatique dans la version 2.0. Les fichiers journaliers s'accumulent indéfiniment dans le répertoire des logs. Un administrateur doit mettre en place sa propre politique de rotation pour archiver ou supprimer les vieux logs. Une évolution future pourrait ajouter une configuration pour supprimer automatiquement les logs plus vieux qu'un nombre de jours spécifié. Cette fonctionnalité éviterait l'accumulation infinie de fichiers tout en laissant le contrôle à l'utilisateur sur la durée de rétention.

L'analyse des logs permet de générer des statistiques utiles. En parcourant les fichiers log, on peut calculer le nombre total de fichiers sauvegardés, la quantité totale de données transférées, la durée moyenne de transfert par fichier, les travaux les plus volumineux, les heures de pointe. Ces analyses peuvent être effectuées avec des scripts externes qui parsent les fichiers JSON ou XML. Le format structuré facilite ce type de traitement automatisé. Une évolution future pourrait intégrer un tableau de bord dans l'interface affichant ces statistiques directement.

4.5 Gestion du fichier d'état

Le fichier d'état maintient l'information en temps réel sur tous les travaux de sauvegarde. Contrairement aux fichiers log qui enregistrent l'historique, le fichier d'état reflète la situation actuelle. Cette distinction permet aux applications externes de surveiller les sauvegardes en cours sans avoir à parser l'intégralité de l'historique.

La structure du fichier d'état utilise un unique fichier JSON nommé state.json par défaut. Ce fichier contient un objet JSON où chaque clé est le nom d'un travail et chaque valeur est un objet StateEntry décrivant l'état actuel de ce travail. Cette structure en dictionnaire permet un accès direct à l'état d'un travail spécifique en utilisant son nom comme clé. La mise à jour d'un travail n'affecte que son entrée sans toucher aux autres travaux.

Chaque entrée d'état contient neuf champs. Le champ "jobName" répète le nom du travail pour faciliter le parsing. Le champ "timestamp" enregistre l'horodatage de la dernière mise à jour au format ISO 8601 en UTC. Ce champ permet de détecter si l'état est récent ou obsolète. Le champ "state" contient l'état actuel du travail parmi cinq valeurs possibles : "PENDING" pour un travail créé mais jamais exécuté, "ACTIVE" pour un travail en cours d'exécution, "COMPLETED" pour un travail terminé avec succès, "PAUSED" pour un travail en pause, "ERROR" pour un travail qui a rencontré une erreur fatale. Le champ "totalFiles" contient le nombre total de fichiers à traiter. Ce nombre est calculé au début de l'exécution en parcourant tous les répertoires source. Le champ "totalSize" contient la taille totale en octets de tous les fichiers à traiter. Le champ "progress" contient le pourcentage de progression entre 0 et 100. Ce pourcentage se calcule en divisant la taille déjà traitée par la

taille totale. Le champ "remainingFiles" contient le nombre de fichiers restant à traiter. Le champ "remainingSize" contient la taille en octets restant à traiter. Les champs "currentSourceFile" et "currentTargetFile" contiennent les chemins UNC du fichier actuellement en cours de traitement. Ces champs sont vides quand aucun traitement n'est en cours.

La mise à jour du fichier d'état s'effectue en temps réel durant l'exécution. Le StateWriter est appelé à plusieurs moments clés. Au début de l'exécution d'un travail, l'état passe à "ACTIVE" avec la progression à 0. Après chaque fichier traité, la progression se recalcule et l'état se met à jour. À la fin de l'exécution, l'état passe à "COMPLETED" avec la progression à 100. Si une erreur fatale survient, l'état passe à "ERROR". Chaque mise à jour réécrit le fichier complet. Cette approche garantit que le fichier reste toujours cohérent. Un verrou protège l'accès au fichier pour la thread-safety.

La fréquence de mise à jour représente un compromis entre la fraîcheur des données et les performances. Mettre à jour après chaque fichier copié garantit que l'état est toujours à jour mais génère beaucoup d'écritures disque pour les travaux avec de nombreux petits fichiers. Une approche par batch qui ne met à jour que toutes les 10 secondes réduit les écritures mais retarde l'information. La version 2.0 choisit de mettre à jour après chaque fichier car la bibliothèque EasyLog gère déjà l'écriture de manière efficace et les tests montrent que l'impact sur les performances est négligeable.

Les applications externes peuvent surveiller le fichier d'état de plusieurs manières. La méthode la plus simple consiste à lire le fichier régulièrement, par exemple toutes les secondes, parser le JSON et afficher les informations pertinentes. Cette approche fonctionne bien pour un dashboard web ou une application de monitoring. Une méthode plus sophistiquée utilise un FileSystemWatcher qui détecte automatiquement les modifications du fichier et déclenche un traitement. Cette approche réactive évite de poller contre le fichier inutilement. Les deux approches sont valides et le choix dépend des contraintes de l'application externe.

La compatibilité avec les applications tierces est garantie par la stabilité du format. La structure du fichier d'état est documentée et ne change pas entre les versions mineures. Une application qui parse correctement le format de la version 2.0 continuera de fonctionner avec les versions 2.1, 2.2, etc. Les versions majeures futures pourraient ajouter des champs supplémentaires mais conserveront les champs existants pour la rétrocompatibilité. Un

parser externe qui ignore les champs inconnus continuera de fonctionner même avec un fichier généré par une version plus récente.

Le nettoyage des entrées obsolètes s'effectue manuellement ou via l'interface. Quand un travail est supprimé de l'application, son entrée est retirée du fichier d'état. Cette opération garantit que le fichier ne contient que des travaux actifs. Si l'application se termine brutalement sans supprimer proprement ses entrées, elles persistent dans le fichier. Au prochain démarrage, l'application peut détecter et nettoyer ces entrées orphelines. Cette logique de nettoyage évite l'accumulation d'entrées obsolètes dans le fichier d'état.

Les erreurs de lecture ou d'écriture du fichier d'état sont gérées de manière résiliente. Si le fichier est corrompu et ne peut pas être parsé, l'application affiche un avertissement et crée un nouveau fichier vide. L'état précédent est perdu mais l'application reste fonctionnelle. Si l'écriture échoue à cause de permissions insuffisantes, l'erreur est loggée mais n'empêche pas l'exécution de la sauvegarde. Cette approche priorise la fonction principale de sauvegarde sur la fonction secondaire de monitoring d'état.

La localisation du fichier d'état est configurable dans les paramètres. Par défaut, le fichier se situe dans le répertoire AppData de l'utilisateur. Un administrateur peut spécifier un chemin différent, par exemple sur un partage réseau accessible par plusieurs systèmes. Cette flexibilité permet de centraliser les fichiers d'état de plusieurs instances d'EasySave pour un monitoring centralisé. Néanmoins, cette approche nécessite de gérer les conflits si plusieurs instances tentent d'écrire simultanément dans le même fichier. La version 2.0 ne gère pas automatiquement ce scénario qui reste rare en pratique.

5. Diagrammes UML

5.1 Diagramme de classes

Le diagramme de classes du livrable 2 met en évidence l'évolution d'EasySave vers une application graphique reposant sur une architecture MVVM, tout en conservant une séparation stricte entre la logique métier, les services applicatifs et la couche de présentation. La classe BackupJob constitue l'entité centrale du système et regroupe l'ensemble des informations nécessaires à la gestion et au suivi d'un travail de sauvegarde, incluant désormais l'état du travail, la progression, ainsi que les paramètres liés au cryptage. La classe Configuration, accessible via le ConfigurationManager implémenté en singleton, centralise les paramètres globaux tels que la langue, le format des logs ou les applications bloquantes, garantissant une cohérence du comportement de l'application.

La logique métier est orchestrée par le BackupManager, qui implémente un rôle de gestionnaire central des travaux de sauvegarde et délègue l'exécution effective au FileTransferService. Ce service assure le transfert des fichiers, la mise à jour du fichier d'état via le StateWriter, et la journalisation via l'interface ILogger, tout en intégrant le CryptageManager pour appliquer le cryptage si nécessaire. Les objets BackupLogEntry et StateEntry structurent respectivement les informations de journalisation et l'état temps réel, garantissant une traçabilité complète et conforme au cahier des charges.

La bibliothèque externe EasyLog.dll constitue un composant autonome chargé de la journalisation. Elle repose sur l'interface ILogger, implémentée par des loggers spécialisés tels que JsonLogger et XmlLogger, permettant de supporter plusieurs formats sans modifier la logique métier. Le mécanisme de formatage repose sur l'interface générique ILogFormatter, implémentée par des formateurs spécifiques, garantissant une extensibilité du système. Cette séparation assure l'indépendance entre le moteur de sauvegarde et la stratégie de journalisation, facilitant la maintenance et l'évolution du logiciel.

Enfin, la couche de présentation repose sur le pattern MVVM, avec des ViewModels tels que MainViewModel, BackupJobViewModel et SettingsViewModel, qui assurent l'interaction entre l'interface graphique et les services métiers. Les vues, telles que MainWindow, sont découplées de la logique applicative et se limitent à l'affichage et aux interactions utilisateur.

5.2 Diagramme de cas

Le diagramme de cas d'utilisation de la version 2.0 met en évidence l'évolution du système EasySave vers une application graphique, tout en conservant les fonctionnalités fondamentales du moteur de sauvegarde. L'utilisateur reste l'acteur principal du système, mais ses interactions s'effectuent désormais à travers une interface graphique, ce qui améliore l'ergonomie et la lisibilité des opérations.

Le diagramme formalise l'ensemble des actions possibles, incluant la création, la modification, la suppression et l'exécution des travaux de sauvegarde. Il introduit également la gestion des paramètres globaux du logiciel, permettant à l'utilisateur de définir des préférences telles que la langue ou le format des journaux. Cette fonctionnalité centralisée garantit une cohérence de fonctionnement entre les différentes sessions et prépare l'extension vers des configurations plus avancées dans les versions futures.

La consultation des journaux d'événements constitue également un cas d'utilisation explicite, ce qui permet à l'utilisateur d'accéder directement à l'historique des opérations depuis l'interface. Cette fonctionnalité améliore la traçabilité et facilite les opérations de diagnostic et de support. Le diagramme montre que la journalisation reste un mécanisme transversal, automatiquement déclenché lors de l'exécution d'une sauvegarde, sans intervention spécifique de l'utilisateur.

Cette modélisation confirme que l'introduction de l'interface graphique n'altère pas la logique métier existante, mais constitue une nouvelle couche de présentation reposant sur les mêmes services internes, conformément à l'architecture modulaire définie lors de la version 1.0.

5.3 Diagramme de séquence

Les diagrammes de séquence décrivent les interactions dynamiques entre les différents composants du système lors des opérations principales, en mettant en évidence le rôle central de la couche de services et la séparation entre l'interface graphique et la logique métier.

Le diagramme correspondant à la création d'un travail de sauvegarde montre que l'interface utilisateur transmet les informations saisies au gestionnaire de sauvegarde, qui valide les données et les enregistre dans le fichier de configuration. Le système confirme ensuite l'opération en actualisant l'interface, garantissant la cohérence entre la vue et l'état interne de l'application.

Le diagramme relatif à la modification d'un travail suit un processus similaire. Les nouvelles informations sont transmises au gestionnaire, qui met à jour la configuration persistée. Cette approche garantit que toutes les modifications sont centralisées et validées avant leur application, assurant l'intégrité du système.

Le diagramme d'exécution d'un travail de sauvegarde met en évidence l'enchaînement complet des opérations. L'interface déclenche la demande d'exécution, qui est transmise au gestionnaire de sauvegarde. Celui-ci orchestre le transfert des fichiers via le service dédié, tout en mettant à jour l'état en temps réel et en enregistrant les événements dans le système de journalisation. Cette séquence confirme la séparation entre la couche de présentation et les mécanismes internes de traitement.

Le diagramme de suppression d'un travail illustre le processus de retrait d'une tâche existante. L'interface transmet la demande au gestionnaire, qui supprime l'entrée correspondante et met à jour la configuration persistée. L'interface est ensuite actualisée afin de refléter l'état réel du système.

Enfin, le diagramme relatif à la gestion des paramètres montre que l'interface transmet les modifications au gestionnaire de configuration, qui valide et enregistre les nouvelles valeurs. Ces paramètres sont ensuite utilisés par les différents composants du système, garantissant une configuration centralisée et cohérente.

Ces diagrammes confirment que la version 2.0 repose sur une architecture respectant les principes de séparation des responsabilités et d'inversion de dépendances. L'interface graphique agit uniquement comme un intermédiaire entre l'utilisateur et les services applicatifs, tandis que la logique métier reste isolée et réutilisable, ce qui facilite la maintenance et les évolutions futures du logiciel.

5.4 Diagramme d'activité

Les diagrammes d'activité décrivent les différents flux fonctionnels introduits ou enrichis par l'interface graphique de la version 2.0. Ils permettent de formaliser les interactions entre l'utilisateur et le système, ainsi que les étapes internes nécessaires à l'exécution des opérations.

Le diagramme relatif à la création et à la modification d'un travail de sauvegarde illustre le processus de configuration des paramètres essentiels, notamment le nom, les répertoires source et cible, ainsi que le type de sauvegarde. Le système valide les informations fournies avant de les persister, garantissant l'intégrité des données. Cette validation permet d'éviter la création de tâches incohérentes ou incomplètes.

Le diagramme dédié à la consultation et à la sélection des travaux de sauvegarde décrit le mécanisme de récupération des tâches existantes depuis le fichier de configuration et leur affichage dans l'interface graphique. L'utilisateur peut sélectionner un travail afin de consulter ses informations ou déclencher une action spécifique, comme son exécution ou sa modification. Ce flux garantit une interaction claire entre la vue et le moteur applicatif.

Le diagramme d'activité relatif à la consultation des logs journalier formalise l'accès aux fichiers de journalisation générés lors des sauvegardes. Le système lit les fichiers correspondants, interprète leur contenu et les présente de manière exploitable pour l'utilisateur. Cette fonctionnalité permet un suivi précis des opérations et renforce la capacité de diagnostic du logiciel.

Enfin, le diagramme concernant la gestion des paramètres montre le processus de modification des préférences globales. Les paramètres sélectionnés sont validés, puis enregistrés dans le fichier de configuration, assurant leur persistance entre les sessions. Ce mécanisme garantit que l'application conserve un comportement cohérent et configurable selon les besoins de l'utilisateur.

Ces diagrammes confirment que l'introduction de l'interface graphique n'a pas modifié la logique interne du système, mais a permis d'encapsuler les interactions utilisateur dans une couche dédiée, respectant le principe de séparation des responsabilités.

7. Table des figures

Figure 1 : Tableau répertoriant les tâches à réaliser pour le livrable 2.....	17
Figure 2 : Un croquis de la page d'accueil, en encadré bleu la side bar avec un bouton Home en haut et le bouton Settings en bas. En encadré vert, les tâches et un résumé de leur progression, état et informations sources et destinations avec la possibilité de les cocher, il est aussi possible de rechercher les tâches par nom et de les parcourir via "l'ascenseur". En encadré rouge les boutons d'actions permettant de supprimer, exécuter ou ajouter des tâches des sauvegardes.....	17
Figure 3 : Pop-up de création d'une tâche avec spécification des sources (possibilité de sélectionner les sources via explorateur de fichiers avec le crayon), de même pour la destination, les extensions de fichiers à sauvegarder, le type de sauvegarde, l'annulation de la création ou l'exécution de la tâche créée.....	18
Figure 4 : Croquis de l'onglet de sélection des tâches, avec la possibilité de drag n drop les tâches afin de redéfinir l'ordre d'exécution de ces dernières.....	18
Figure 5 : Scénario des différents onglets et fonctionnalités.....	19
Figure 6 : Croquis de la partie des logs de l'application pour que l'utilisateur puisse visualiser les logs au lieu de se rendre dans AppData.....	19
Figure 7 : Extrait du trello pour le livrable 2.....	20
Figure 8 : Tableau de chiffrage du livrable 1 et 2.....	21

8. Glossaire

Avalonia : Framework open-source basé sur .NET permettant de développer des interfaces graphiques multiplateformes (Windows, Linux, macOS) en C# et XAML, avec une approche proche de WPF.

Backend : Partie non visible d'une application contenant la logique métier, les services et les traitements (sauvegarde, journalisation, chiffrement, persistance). Dans EasySave, il correspond principalement aux Managers et Services.

Binding (Data Binding) : Mécanisme reliant automatiquement une propriété d'un composant de l'interface graphique à une propriété d'un ViewModel. Toute modification d'un côté est répercutée de manière transparente.

Changelog / Release Note : Document synthétisant les modifications apportées entre deux versions d'un logiciel (fonctionnalités ajoutées, corrections, améliorations, limitations connues).

CI/CD (Continuous Integration / Continuous Deployment) : Ensemble de pratiques et d'outils visant à automatiser la compilation, les tests et la publication des versions d'un logiciel. Dans le projet, cette chaîne est réalisée via GitHub Actions.

CryptoSoft : Logiciel externe utilisé pour chiffrer certains fichiers selon des extensions configurées. EasySave déclenche CryptoSoft durant la sauvegarde et mesure le temps de chiffrement.

DLL (Dynamic Link Library) : Bibliothèque compilée utilisable par plusieurs applications. EasySave utilise EasyLog.dll pour mutualiser la journalisation et garantir la réutilisabilité de cette fonctionnalité.

DRY (Don't Repeat Yourself) : Principe de conception visant à éviter la duplication de code. L'objectif est de centraliser la logique afin d'améliorer la maintenabilité et de limiter les risques de divergence.

Exception : Mécanisme .NET signalant une erreur au runtime. Les exceptions permettent de remonter un problème (I/O, permissions, fichier manquant) et d'appliquer un traitement adapté.

Framework : Ensemble d'outils, bibliothèques et conventions facilitant le développement logiciel. .NET 8 et Avalonia sont des frameworks utilisés dans le projet.

GitFlow : Modèle d'organisation Git basé sur des branches structurées, généralement **main** (stable), **develop** (intégration), et des branches de fonctionnalités ou correctifs.

GUI (Graphical User Interface) : Interface graphique permettant d'utiliser l'application via des éléments visuels (fenêtres, boutons, formulaires), par opposition à une interface console.

ICommand : En Avalonia, ICommand sert à gérer les actions de l'interface (clic bouton, menu, raccourci clavier, etc...) dans le pattern MVVM.

ILogger : Interface définissant un contrat de journalisation (écriture et lecture de logs). Elle permet de découpler le moteur de sauvegarde du format réel des logs.

Injection de dépendances (Dependency Injection) : Technique consistant à fournir les dépendances d'une classe depuis l'extérieur (constructeur), plutôt que de les instancier en interne. Elle réduit le couplage et facilite les tests.

INotifyPropertyChanged : En Avalonia, INotifyPropertyChanged sert à prévenir l'interface graphique qu'une valeur a changé dans le ViewModel.

JSON (JavaScript Object Notation) : Format de données textuel structuré, léger et lisible. Utilisé pour les fichiers de configuration, d'état et de logs dans EasySave.

Log (journalisation) : Enregistrement chronologique des actions réalisées par l'application (transfert d'un fichier, erreur, arrêt, etc.). Les logs servent au support technique et à l'audit des opérations.

MVVM (Model–View–ViewModel) : Architecture séparant les données (Model), l'interface graphique (View) et la logique de présentation (ViewModel). MVVM facilite le test, la maintenance et l'évolution de l'interface.

ObservableCollection : Collection .NET conçue pour les interfaces graphiques. Elle notifie automatiquement l'UI lorsqu'un élément est ajouté, supprimé ou modifié.

Patch / Minor / Major : Niveaux de versioning sémantique. Major implique une rupture de compatibilité, Minor ajoute des fonctionnalités compatibles, Patch corrige des bugs sans impact fonctionnel.

Repository Pattern : Patron de conception encapsulant l'accès aux données (lecture/écriture) derrière une interface stable. Dans EasySave, le BackupManager agit comme un point central pour persister et récupérer les jobs.

Singleton : Patron de conception garantissant qu'une classe ne possède qu'une seule instance durant l'exécution. Utilisé pour la configuration afin d'assurer une cohérence globale des paramètres.

State (fichier d'état) : Fichier mis à jour en temps réel décrivant l'avancement des travaux (progression, fichier en cours, taille restante, etc.). Il permet la supervision externe ou l'affichage dynamique dans l'UI.

Thread-safe : Qualifie un composant capable de fonctionner correctement même en cas d'accès concurrent (plusieurs appels simultanés). Les écritures sur les fichiers de logs/état sont sécurisées via des mécanismes de verrouillage.

UNC (Universal Naming Convention) : Format standard de chemin réseau Windows permettant d'identifier un fichier ou un partage via une syntaxe du type \\Serveur\Partage\.... Garantit la compatibilité avec les lecteurs réseau.

WPF (Windows Presentation Foundation) : Framework graphique Microsoft destiné aux applications desktop Windows, basé sur XAML et très compatible avec MVVM. Mentionné dans l'étude comparative des frameworks.

XAML : Langage de description d'interface utilisé pour définir les vues graphiques (mise en page, styles, composants). Avalonia exploite le XAML pour construire l'UI.

%AppData% : Répertoire Windows prévu pour stocker des données applicatives persistantes liées à un utilisateur. EasySave l'utilise pour les fichiers de configuration, d'état et de journalisation.

9. Annexes

9.1 Guide d'installation et Support Technique (Détails ProSoft)

cf. Manuel_EasySave_v1.0.pdf

9.2 Release note EasySave

cf. Release_Note_EasySave_v1.1.pdf

cf. Release_Note_EasySave_v2.0.pdf

9.3 Lien vers le dépôt GitHub

<https://github.com/MatheoCarval/Projet-EasySave>